

Trading Research Platform

Complete Technical Documentation

Signal Generation · Backtesting Engine · Performance Metrics · Risk Management

This document explains every component of the platform in precise detail: how each trading strategy generates signals, how look-ahead bias is prevented, how the backtest engine simulates trades bar by bar, how position sizes are calculated using volatility, how risk is managed with stop-losses and take-profits, and what every performance metric measures.

Table of Contents

§	Section	Page
1	Architecture Overview	3
2	Data Ingestion	4
3	Signal Convention & Look-Ahead Bias	5
4	Trading Strategies	6
4.1	SMA Crossover	6
4.2	RSI (Relative Strength Index)	7
4.3	Stochastic RSI	9
4.4	MACD	10
4.5	Bollinger Bands	12
4.6	Donchian Channels	13
4.7	ADX (Average Directional Index)	14
4.8	Ichimoku Cloud	16
5	Signal Combination	18
6	Higher-Timeframe Trend Filter	20
7	Backtest Engine	21
7.1	Bar-by-Bar Simulation Loop	21
7.2	Volatility-Based Stop-Loss Sizing	23
7.3	Position Sizing	25
7.4	Take-Profit Calculation	26
7.5	Commission & Slippage	26
7.6	Signal Re-Entry (Pending Entry)	27
8	Performance Metrics	28
9	Trade Log Columns	33
10	Parameter Sweep	34

1. Architecture Overview

The platform is structured as five independent Python modules that communicate through well-defined interfaces. Each module has a single responsibility and imports nothing from the others except where explicitly stated. This makes each piece easy to test, extend, or replace in isolation.

Module	File	Responsibility
Data	data.py	Downloads OHLCV price data from Yahoo Finance via yfinance and caches it in the Streamlit session.
Strategies	strategies.py	Implements all 8 trading strategies plus signal combination and the higher-timeframe trend filter.
Backtest	backtest.py	Bar-by-bar simulation engine: converts signals to positions, applies SL/TP, computes returns and all metrics.
Utils	utils.py	Shared helpers: return calculation, annualisation factor, all Plotly charting functions.
App	app.py	Streamlit UI entry point. Wires all modules together into a three-tab web application.

Data flow (left to right):

- **data.py** fetches OHLCV price bars from Yahoo Finance.
- **strategies.py** reads the price DataFrame and produces a signal Series (1, -1, or 0 per bar).
- **backtest.py** reads both the signal Series and the price DataFrame, simulates trades bar by bar, and returns metrics, an equity curve, and a trade log.
- **utils.py** converts those outputs into Plotly charts.
- **app.py** orchestrates the above in order, stores results in Streamlit session state between re-renders, and displays everything across three tabs.

2. Data Ingestion

All market data is sourced from Yahoo Finance through the **yfinance** library. The function **fetch_data(ticker, interval, start, end)** in **data.py** handles downloading, cleaning, and caching.

Supported Timeframes

The platform supports five bar intervals:

Interval	Bar Duration	Yahoo Finance Limit	Approx. Bars per Year
1d	1 calendar day	No limit	252
4h	4 hours	730 days	~411
1h	1 hour	730 days	~1,638
15m	15 minutes	60 days	~1,638
5m	5 minutes	60 days	~4,914

The Yahoo Finance API enforces hard limits on how far back intraday data goes: 5-minute and 15-minute data is available for the last 60 days only; 1-hour and 4-hour data is available for the last 730 days only. The platform enforces these limits before attempting a download and shows an error to the user if the requested date range exceeds the limit.

auto_adjust=True

All downloads use **auto_adjust=True**. This instructs Yahoo Finance to return prices already adjusted for historical stock splits and dividend distributions. Without adjustment, a 2:1 stock split would appear in the price history as a 50% crash overnight — the backtest engine would then generate a massive sell signal and record a large loss that never actually happened. Adjusted prices ensure the entire price history is internally consistent and suitable for accurate backtesting.

Session Caching

Downloaded data is stored in **st.session_state** using a cache key of the form *ticker_interval_start_end*. If the user clicks 'Run Backtest' again with identical settings, the cached DataFrame is returned immediately without a second network request. The cache is invalidated whenever the ticker, interval, or date range changes.

Data Cleaning

- If yfinance returns a MultiIndex column structure (which can occur on some versions), it is flattened to standard column names.
- Only the five standard OHLCV columns are retained: Open, High, Low, Close, Volume.
- Any row where Close is NaN is dropped — this can occur at the end of the data range when the most recent bar has not yet closed.
- A minimum of 100 bars is required before a backtest can run. Below this threshold the platform rejects the data with an error message.

3. Signal Convention & Look-Ahead Bias Prevention

Signal Values

Every strategy returns a `pd.Series` of integer values with the same index as the price `DataFrame`. The three possible values are:

Value	Meaning
1	Long signal — go long (buy) at the next bar's open.
-1	Short signal — go short (sell) at the next bar's open.
0	No signal — do nothing, stay flat.

Event-Based Signals

Signals are **event-based**, not state-based. A signal value of 1 means *a crossover just happened on this bar*, not *the fast line is currently above the slow line*. A signal fires on exactly one bar — the bar where the condition transitions from false to true — and is 0 on every other bar, even if the condition continues to hold. This is implemented using pandas shift comparisons:

```
signal[(fast > slow) & (fast.shift(1) <= slow.shift(1))] = 1
```

The left condition checks the current bar; the right condition checks the previous bar. A signal fires only when the current bar satisfies the condition AND the previous bar did not — i.e. only on the exact bar of the transition.

The One-Bar Delay Rule (Look-Ahead Prevention)

Look-ahead bias is the most common mistake in backtesting. It occurs when the backtest uses information from the current bar to decide whether to enter a trade on the same bar. In reality, you cannot observe the close price of bar N and simultaneously enter a trade on bar N — the close price is the last price of the bar, by which time the bar is over.

The platform prevents look-ahead bias by applying a **one-bar shift** to all signals inside the backtest engine:

```
shifted_signals = signals.shift(1).reindex(prices.index).fillna(0)
```

This means: a signal generated at the close of bar N is only acted upon at bar N+1. The position is entered at **the close of bar N** (used as the entry price proxy), and the trade result is calculated from bar N+1 onwards. This correctly models the real-world workflow: observe the close, decide to trade, execute at or near the next open.

This is a non-negotiable design decision. Any backtest without this shift is technically invalid and will report unrealistically good results.

4. Trading Strategies

4.1 SMA Crossover

Type: Trend-following | **Parameters:** *fast_period* (default 10), *slow_period* (default 50)

The Simple Moving Average crossover is one of the oldest and most widely used systematic trading signals. It is based on the idea that when a short-term average of price crosses above a long-term average, momentum has shifted upward and a trend may be beginning.

How it is calculated

- **Fast SMA:** the arithmetic mean of the closing price over the last *fast_period* bars. By default this is 10 bars.
- **Slow SMA:** the arithmetic mean of the closing price over the last *slow_period* bars. By default this is 50 bars.

Formula:

$$\text{SMA}(N) \text{ at bar } t = (\text{Close}[t] + \text{Close}[t-1] + \dots + \text{Close}[t-N+1]) / N$$

Signal logic

The signal fires only on the bar where the crossover actually happens — not on every bar where the fast is above the slow:

Long (+1): fast crosses ABOVE slow $\rightarrow \text{fast}[t] > \text{slow}[t] \text{ AND } \text{fast}[t-1] \leq \text{slow}[t-1]$

Short (-1): fast crosses BELOW slow $\rightarrow \text{fast}[t] < \text{slow}[t] \text{ AND } \text{fast}[t-1] \geq \text{slow}[t-1]$

Why it works (theory)

A short-term average reacts quickly to recent price changes. A long-term average moves slowly and reflects the broader trend. When the short-term average crosses above the long-term average, it means recent prices have risen sharply relative to the longer history — a bullish momentum signal. The reverse crossover indicates that recent prices have declined sharply, a bearish signal.

Practical considerations

- **Trending markets:** SMA crossover performs best in clear, sustained trends. It is a momentum/trend-following strategy by design.
- **Ranging markets:** In sideways, choppy markets the two SMAs cross back and forth frequently, generating many small losses known as 'whipsaws'. The ADX strategy (Section 4.7) was designed specifically to address this.
- **Lag:** Moving averages are lagging indicators. The signal always fires some time after the trend has already started. Shorter periods reduce lag but increase whipsaws; longer periods reduce whipsaws but enter later into moves.
- **Parameter constraints:** *fast_period* must be strictly less than *slow_period*. The platform raises an error if this constraint is violated.

4.2 RSI — Relative Strength Index

Type: Mean-reversion | **Parameters:** period (default 14), oversold (default 30), overbought (default 70)

The RSI is a momentum oscillator developed by J. Welles Wilder in 1978. It measures the speed and magnitude of recent price changes to evaluate whether an asset is overbought (too expensive, likely to fall) or oversold (too cheap, likely to rise). RSI always oscillates between 0 and 100.

How RSI is calculated

Step 1: For each bar, compute the price change from the previous bar:

```
Delta[t] = Close[t] - Close[t-1]
```

Step 2: Separate gains and losses:

```
Gain[t] = max(Delta[t], 0) Loss[t] = max(-Delta[t], 0)
```

Step 3: Compute smoothed average gain and average loss over the lookback period (the ta library uses Wilder's exponential smoothing, which gives more weight to recent values than a simple average):

```
Avg Gain = Exponential Moving Average of Gain over period bars
```

```
Avg Loss = Exponential Moving Average of Loss over period bars
```

Step 4: Compute the Relative Strength (RS) and then the RSI:

```
RS = Avg Gain / Avg Loss
```

```
RSI = 100 - (100 / (1 + RS))
```

When all recent bars closed higher, Avg Loss approaches zero, RS approaches infinity, and RSI approaches 100. When all recent bars closed lower, Avg Gain approaches zero, RS approaches zero, and RSI approaches 0.

Signal logic

The platform uses a **reversal confirmation** approach rather than a simple threshold level. This means the signal fires when RSI exits the extreme zone, not when it enters it:

```
Long (+1): RSI crosses back UP through the oversold level
```

```
prev_RSI < oversold AND RSI >= oversold
```

```
Short (-1): RSI crosses back DOWN through the overbought level
```

```
prev_RSI > overbought AND RSI <= overbought
```

This confirmation step is important. Entering when RSI first drops below 30 can mean buying into a market that is still falling hard. Waiting for the exit from the oversold zone (the bounce back above 30) provides evidence that the selling pressure has abated and the price is actually starting to recover.

Why it works (theory)

RSI is a mean-reversion strategy. The underlying assumption is that markets tend to overshoot — they fall too far (fear/panic) or rise too far (greed/euphoria) and then revert toward fair value. When RSI reaches extreme levels it indicates that recent price movement has been unusually one-sided. The strategy bets that this imbalance will correct.

Practical considerations

- **Trending markets:** RSI performs poorly in strong trends. In a bull trend, RSI can remain overbought (above 70) for extended periods, generating premature short signals. The higher-TF trend filter (Section 6) can mitigate this.
- **Oversold does not mean 'buy immediately':** An RSI of 10 means the market has fallen sharply; it doesn't mean it can't fall further. The reversal confirmation signal (waiting for the exit from the zone) adds a layer of confirmation.
- **Period sensitivity:** A shorter period (e.g. 7) makes RSI more reactive and oscillate more frequently; a longer period (e.g. 21) makes it smoother and generates fewer, slower signals.

4.3 Stochastic RSI

Type: Mean-reversion | **Parameters:** *period (14), smooth_k (3), smooth_d (3), oversold (20), overbought (80)*

Stochastic RSI applies the Stochastic Oscillator formula to the RSI values rather than directly to price. It was developed by Tushar Chande and Stanley Kroll in 1994. The result is a more sensitive indicator that oscillates between 0 and 100 and reaches extreme levels more frequently than plain RSI.

How it is calculated

Step 1: Compute the RSI series over period bars (same as Section 4.2).

Step 2: Apply the Stochastic formula to the RSI series over the same period:

```
StochRSI = (RSI[t] - min(RSI, period)) / (max(RSI, period) - min(RSI, period))
```

This rescales the RSI so that its highest value over the period equals 1 and its lowest value equals 0.

Step 3: Smooth the raw StochRSI with a rolling mean to get the %K line:

```
%K = rolling_mean(StochRSI, smooth_k)
```

Step 4: Smooth %K again to get the %D (signal) line:

```
%D = rolling_mean(%K, smooth_d)
```

The platform scales %K to 0–100 to match the standard charting convention. The thresholds (default 20 for oversold, 80 for overbought) also follow this 100-point scale.

Signal logic

Identical to RSI: signals fire when %K exits the extreme zones (reversal confirmation):

```
Long (+1): %K crosses up through oversold → prev_%K < 20 AND %K >= 20
```

```
Short (-1): %K crosses down through overbought → prev_%K > 80 AND %K <= 80
```

How it differs from plain RSI

Because StochRSI is RSI-of-RSI, it is much more sensitive and volatile than plain RSI. It reaches extreme levels frequently, generating more signals. The double smoothing (*smooth_k* and *smooth_d*) controls how reactive or smooth the indicator is. Increasing *smooth_k* or *smooth_d* reduces signal frequency and removes short-term noise.

4.4 MACD — Moving Average Convergence Divergence

Type: Trend-following / momentum | **Parameters:** fast (12), slow (26), signal_period (9)

MACD was developed by Gerald Appel in the late 1970s. It works by measuring the relationship between two exponential moving averages of price, and then smoothing that relationship further. It is one of the most widely used indicators in systematic trading.

How it is calculated

Step 1: Compute two Exponential Moving Averages (EMA) of the closing price. Unlike a simple moving average which weights all bars equally, an EMA gives progressively more weight to recent prices:

```
EMA multiplier k = 2 / (N + 1)

EMA[t] = Close[t] * k + EMA[t-1] * (1 - k)
```

Step 2: Compute the MACD line as the difference between the fast and slow EMAs:

```
MACD Line = EMA(fast) - EMA(slow)
```

When the fast EMA is above the slow EMA, MACD is positive (bullish momentum). When it is below, MACD is negative (bearish momentum).

Step 3: Compute the Signal line — an EMA of the MACD line itself:

```
Signal Line = EMA(MACD Line, signal_period)
```

The Signal line is a smoothed version of the MACD line. It reacts more slowly to changes in the MACD.

Step 4: Compute the Histogram — the difference between MACD and Signal:

```
Histogram = MACD Line - Signal Line
```

The histogram is shown as bars on the chart. Positive (green) bars indicate the MACD is above its signal line; negative (red) bars indicate it is below. Growing bars mean momentum is accelerating; shrinking bars mean it is fading.

Signal logic

The platform uses the MACD/Signal line crossover:

```
Long (+1): MACD crosses ABOVE Signal
prev_MACD < prev_Signal AND MACD >= Signal

Short (-1): MACD crosses BELOW Signal
prev_MACD > prev_Signal AND MACD <= Signal
```

Why it works (theory)

MACD is fundamentally a momentum indicator. It captures the relationship between short-term and long-term price momentum. When the MACD line crosses above the signal line, it means short-term momentum is accelerating relative to its own recent history — a bullish signal. The signal line acts as a smoother, confirming that the MACD movement is sustained rather than a one-bar spike.

Practical considerations

4.5 Bollinger Bands

Type: Mean-reversion | **Parameters:** *period* (default 20), *std_dev* (default 2.0)

Bollinger Bands were developed by John Bollinger in the 1980s. They place an upper and lower envelope around price at a configurable number of standard deviations from a moving average. The bands automatically widen during volatile periods and contract during quiet periods.

How it is calculated

Step 1: Compute the Middle Band — a simple moving average of closing price:

```
Middle Band = SMA(Close, period)
```

Step 2: Compute the rolling standard deviation of closing price over the same period:

```
 $\sigma = \sqrt{(1/N) * \sum (Close[i] - SMA)^2)}$  over the last N bars
```

Step 3: Place the Upper and Lower Bands at a multiple of σ from the Middle Band:

```
Upper Band = Middle Band + std_dev *  $\sigma$ 
```

```
Lower Band = Middle Band - std_dev *  $\sigma$ 
```

With *std_dev*=2.0 (default), statistically approximately 95% of all closing prices should fall inside the bands — assuming normally distributed returns.

Signal logic

The platform uses a mean-reversion approach: buy when price touches the lower band (oversold), sell when price touches the upper band (overbought). Signals fire only on the bar price first crosses the band:

```
Long (+1): Close crosses DOWN through Lower Band
```

```
Close[t] <= Lower[t] AND Close[t-1] > Lower[t-1]
```

```
Short (-1): Close crosses UP through Upper Band
```

```
Close[t] >= Upper[t] AND Close[t-1] < Upper[t-1]
```

Why it works (theory)

The underlying assumption is that price tends to revert to its mean after unusual deviations. A price touching the lower band means it has moved more than two standard deviations below its recent average — statistically uncommon if returns were truly random. The strategy bets that this is a temporary overextension that will correct.

Practical considerations

- **Volatile moves:** In a genuine crash or breakout, price can 'walk the band' — touching the outer band for many consecutive bars while a strong trend continues. The mean-reversion assumption breaks down in these conditions.
- **Band width as a volatility gauge:** Narrow bands mean recent volatility has been low; wide bands mean high volatility. Some traders use this to size positions or adjust other indicators.
- **std_dev parameter:** Wider bands (e.g. 2.5–3.0) are touched less often, generating fewer but potentially higher-quality signals. Narrower bands (1.5–2.0) are touched more frequently.

4.6 Donchian Channels

Type: Breakout / momentum | **Parameters:** period (default 20)

Donchian Channels were invented by Richard Donchian, a pioneer of trend following in the 1950s. The famous Turtle Traders system used a variant of this strategy in the 1980s. The logic is simple: if price breaks out to a new N-bar high, a new trend may be starting upward; if it breaks to a new N-bar low, a new trend may be starting downward.

How it is calculated

Upper Channel = Highest High over the last period bars

Lower Channel = Lowest Low over the last period bars

Middle Channel = (Upper + Lower) / 2

Signal logic

A breakout occurs on the first bar that closes above/below the channel boundary. Crucially, the channel is shifted by one bar to avoid look-ahead bias — the comparison is against the channel calculated using only prior bars:

Long (+1): Close > Upper Channel of the PREVIOUS bar (first bar to break out)

Short (-1): Close < Lower Channel of the PREVIOUS bar (first bar to break out)

Signals fire only on the first bar of the breakout. Subsequent bars where price remains outside the channel do not generate new signals.

Why it works (theory)

The Donchian system is purely trend-following. The hypothesis is that if price breaks through the highest level it has seen in the past N bars, the market has shown genuine momentum in that direction and a new trend is likely underway. The strategy catches large moves in exchange for accepting many small losses from false breakouts.

Practical considerations

- **Breakout failure:** Many breakouts fail — price pokes above the channel, triggers a long signal, then reverses back inside. The fixed-ratio stop-loss in the backtest engine limits the damage from these false breakouts.
- **Period length:** A 20-bar period captures roughly one month of daily moves. Longer periods (50–100) filter out shorter-term noise and produce fewer, larger trades. Shorter periods (10–15) are more reactive but generate more false breakouts.
- **Best in trending markets:** Like all momentum strategies, Donchian performs best when trends are sustained. It performs poorly in choppy, range-bound markets.

4.7 ADX — Average Directional Index

Type: Trend filter + DI crossover | **Parameters:** period (default 14), threshold (default 25.0)

The ADX system was also developed by J. Welles Wilder in 1978. It is unique among the strategies in the platform because it explicitly measures whether the market is trending at all before taking a position. An ADX below the threshold means the market is ranging — the strategy goes flat. An ADX above the threshold means a trend is in progress — the strategy then uses the Directional Indicator (+DI/-DI) crossover to determine direction.

How it is calculated

Step 1: For each bar, compute the True Range (TR) — the largest of three measures:

$$TR[t] = \max(High[t] - Low[t], |High[t] - Close[t-1]|, |Low[t] - Close[t-1]|)$$

True Range accounts for overnight gaps — if the market gaps up significantly, the High-Low range of the new bar understates the actual price movement.

Step 2: Compute the Directional Movements:

$$+DM[t] = High[t] - High[t-1] \text{ if positive and } > (Low[t-1] - Low[t]), \text{ else } 0$$
$$-DM[t] = Low[t-1] - Low[t] \text{ if positive and } > (High[t] - High[t-1]), \text{ else } 0$$

Step 3: Smooth TR, +DM, and -DM using Wilder's exponential smoothing over period bars.

Step 4: Compute the Directional Indicators:

$$+DI = 100 * (Smoothed +DM / Smoothed TR)$$
$$-DI = 100 * (Smoothed -DM / Smoothed TR)$$

+DI measures upward pressure; -DI measures downward pressure. When +DI > -DI, the market is being pushed upward; when -DI > +DI, downward.

Step 5: Compute ADX — the smoothed absolute difference between +DI and -DI, normalised by their sum:

$$DX[t] = 100 * | +DI - -DI | / (+DI + -DI)$$
$$ADX[t] = Wilder_EMA(DX, period)$$

ADX ranges from 0 to 100. A rising ADX means the trend (in either direction) is strengthening. Values below ~20–25 indicate a ranging market; values above 25 indicate a trending market; values above 40–50 indicate a very strong trend.

Signal logic

The strategy ONLY generates signals when ADX > threshold (market is trending):

Long (+1): ADX > threshold AND +DI crosses ABOVE -DI

Short (-1): ADX > threshold AND -DI crosses ABOVE +DI

When ADX < threshold the strategy outputs 0 (flat) regardless of what the DI lines are doing.

Why it works (theory)

ADX addresses the fundamental weakness of pure trend-following strategies: they generate many whipsaws in sideways markets. By only trading when ADX confirms that a trend is in progress, the strategy avoids many

4.8 Ichimoku Cloud

Type: Trend-following with cloud filter | **Parameters:** *tenkan* (9), *kijun* (26), *senkou_b* (52)

Ichimoku Kinko Hyo (which translates roughly to 'equilibrium chart at a glance') was developed by journalist Goichi Hosoda, who published it in 1969 after decades of research. It is a complete trading system on its own, combining trend direction, momentum, support/resistance, and a confirmation signal into a single chart. The platform implements the most widely used subset: Tenkan/Kijun crossover confirmed by Cloud position.

The five components

Component	Formula	Role
Tenkan-sen (Conversion Line)	$(\text{Highest High} + \text{Lowest Low}) / 2$ over <i>tenkan</i> bars	Short-term equilibrium. Faster-reacting line.
Kijun-sen (Base Line)	$(\text{Highest High} + \text{Lowest Low}) / 2$ over <i>kijun</i> bars	Medium-term equilibrium. Slower-reacting line.
Senkou Span A (Leading Span A)	$(\text{Tenkan} + \text{Kijun}) / 2$, plotted <i>kijun</i> bars forward	Upper edge of bullish cloud.
Senkou Span B (Leading Span B)	$(\text{Highest High} + \text{Lowest Low}) / 2$ over <i>senkou_b</i> bars, plotted <i>kijun</i> bars forward	Lower edge of bullish cloud.
The Cloud (Kumo)	Shaded region between Span A and Span B	Dynamic support/resistance zone. Colour shows bullish (green) or bearish (red) condition.

Cloud shifting in the platform

The `ta` library computes Span A and Span B at bar `T`, but by convention they are displayed at bar `T+kijun` (26 bars into the future). The platform shifts them forward by *kijun* bars so that `cloud_top[T]` represents the cloud actually visible at bar `T` — not a future value. This is essential to avoid look-ahead bias:

```
cloud_top = max(Span_A.shift(kijun), Span_B.shift(kijun))
cloud_bottom = min(Span_A.shift(kijun), Span_B.shift(kijun))
```

Signal logic

The signal requires two conditions to be satisfied simultaneously:

```
Long (+1): Tenkan crosses ABOVE Kijun AND Close > cloud_top
Short (-1): Tenkan crosses BELOW Kijun AND Close < cloud_bottom
```

The Cloud acts as a confirmation filter: long signals are only taken when price is already above the cloud (bullish trend), and short signals only when price is below the cloud (bearish trend). This prevents trading against the prevailing trend.

Why it works (theory)

The Tenkan/Kijun cross is similar in concept to an SMA crossover, but using midpoint averages rather than arithmetic means — this makes the lines more responsive to high and low extremes than to the average close. The Cloud adds a second, independent confirmation layer: even if the fast line crosses the slow line,

5. Signal Combination

When multiple strategies are selected, their individual signals must be combined into a single composite signal before the backtest runs. Three combination methods are available, each representing a different consensus philosophy.

Why combine signals?

Each strategy has strengths in different market conditions. Trend-following strategies like SMA and Donchian perform well in sustained trends but suffer in ranges. Mean-reversion strategies like RSI and Bollinger Bands perform well in ranges but suffer in trends. Combining signals from strategies with different philosophies can smooth out the performance profile — when one strategy struggles, another may compensate.

Method 1: Majority Vote

The simplest and most robust combination method. Each strategy votes +1 (long), -1 (short), or 0 (abstain). The sign of the total vote determines the signal:

`Combined Signal = sign( sum of all individual signals )`

- If more strategies are long (+1) than short (-1), the combined signal is +1 (Long).
- If more are short, the combined signal is -1 (Short).
- If exactly balanced (e.g. one long, one short), the sum is 0 — flat.
- Abstentions (0) from strategies that have no signal count as 0 votes.

Example: 3 strategies selected — SMA=+1, RSI=0, MACD=+1. Sum = +2. Sign(+2) = +1 → Long.

Method 2: Weighted Average

Like majority vote but assigns different importance to each strategy. Weights are entered in the sidebar and normalised internally so they sum to 1:

`Normalised weight(i) = weight(i) / sum(all weights)`

`Combined Signal = sign( sum( normalised_weight(i) * signal(i) ) )`

*Example: SMA weight=2, RSI weight=1, both long (+1). Normalised: SMA=0.667, RSI=0.333. Weighted sum = 0.667*1 + 0.333*1 = 1.0. Sign(1.0) = +1 → Long.*

A strategy with weight 0 is completely ignored. A strategy with a very high weight dominates the combined signal.

Method 3: Threshold

A stricter method that requires stronger consensus before taking a position. The average signal (ranging from -1 to +1) must exceed a threshold:

`Average Signal = mean of all individual signals`

`Long if Average > +threshold`

`Short if Average < -threshold`

Flat otherwise

With threshold=0.5 and three strategies, at least two must agree (and the dissenter votes 0, not against) for a signal to be generated. This is stricter than majority vote. With threshold=0.33, a single strategy signal ($1/3 \approx 0.33$) is sufficient.

The threshold parameter (0.1 to 1.0) is adjustable in the sidebar. Higher values require more consensus; lower values require less.

6. Higher-Timeframe Trend Filter

The trend filter is an optional overlay that suppresses signals on the current timeframe when they conflict with the trend on the next timeframe up. For example, if trading on the 1-hour chart, the filter checks whether the daily trend is bullish or bearish, and blocks short signals when the daily trend is up (and vice versa).

Higher-timeframe mapping

Trading Interval	Higher TF Used for Filter
5 minutes	1 hour
15 minutes	4 hours
1 hour	1 day (daily)
4 hours	1 day (daily)
1 day	1 week (weekly, closing Friday)

Implementation

Rather than making a second API call for higher-TF data, the filter derives it by resampling the existing price data that was already downloaded:

```
htf_close = prices['Close'].resample(rule).last().dropna()
```

This takes the existing bar-by-bar close series and resamples it to the higher timeframe by selecting the last close within each higher-TF period. For example, resampling hourly bars with rule='D' produces one bar per day using the last hourly close of each trading day — equivalent to the daily close.

A simple moving average of configurable period (default 50 bars) is then computed on the higher-TF close:

```
Higher-TF trend = +1 if HTF Close >= HTF SMA(50)
-1 if HTF Close < HTF SMA(50)
```

The resulting trend signal (1 or -1, one value per higher-TF bar) is then forward-filled onto every bar of the original lower-TF signal series:

```
trend_aligned = trend.reindex(signal.index).ffill()
```

This ensures every lower-TF bar has a higher-TF trend value — the current higher-TF bar's trend holds until the next higher-TF bar closes.

Filtering rule

Long signals (+1) are zeroed out when the higher-TF trend is bearish (-1).

Short signals (-1) are zeroed out when the higher-TF trend is bullish (+1).

Flat signals (0) are unchanged regardless of trend direction.

Practical effect

The trend filter converts the underlying strategy into a trend-following regime-aware strategy. It will typically reduce the total number of trades significantly (by removing all counter-trend signals) and will alter the win rate and return profile considerably. It is not always beneficial — mean-reversion strategies like RSI and Bollinger Bands may actually perform worse with the filter applied, since their edge often comes specifically from trading against short-term overextensions within larger trends.

7. Backtest Engine

7.1 Bar-by-Bar Simulation Loop

The core of the backtest engine is a single loop that iterates through every price bar from bar 1 to the end of the data (bar 0 is skipped because the returns calculation requires a prior close). At each bar the engine does exactly what a real trader would have to do — in order, with no knowledge of future bars.

Order of operations at each bar

The following steps happen in this precise order at every bar i :

- **Step 1 — Read current bar data:** Extract `bar_high`, `bar_low`, `bar_close`, and `bar_return` for bar i . Read the shifted signal for bar i (which was generated at bar $i-1$'s close).
- **Step 2 — Check exits (if in a trade):** Before doing anything else, check whether the current bar's High or Low triggered the Stop-Loss or Take-Profit. Also check whether a reversal signal arrived on this bar.
- **Step 3 — Update MAE/MFE tracking:** Update the trade's running minimum low and maximum high with this bar's values.
- **Step 4 — Resolve effective signal:** Determine whether to act on the current bar's signal, a queued `pending_entry` from a prior signal flip, or nothing.
- **Step 5 — Open new trade (if not in a trade):** If a valid signal exists and no trade is open, calculate the SL/TP levels, position size, and open the trade.
- **Step 6 — Check SL/TP on entry bar:** Immediately check whether the new trade's SL or TP was hit on the same bar it was opened. This handles gap-downs/gap-ups on entry.

What each variable tracks

Variable	Type	Purpose
<code>in_trade</code>	bool	True while a position is open.
<code>direction</code>	int	+1 for Long, -1 for Short.
<code>sl_price</code>	float	Absolute stop-loss price level.
<code>tp_price</code>	float	Absolute take-profit price level.
<code>sl_pct_val</code>	float	SL distance as a fraction of entry price (e.g. 0.02 = 2%).
<code>tp_pct_val</code>	float	TP distance as a fraction of entry price (always $2 \times \text{sl_pct_val}$).
<code>pos_fraction</code>	float	Position size as a fraction of total capital (0 to 1.0).
<code>pending_entry</code>	int	Direction of a queued re-entry signal after a signal flip exit.
<code>trade_min_low</code>	float	Lowest Low seen since trade opened (used for MAE/MFE).
<code>trade_max_high</code>	float	Highest High seen since trade opened (used for MAE/MFE).
<code>current_entry_idx</code>	Index	The bar index (timestamp) when the current trade was opened.

Variable	Type	Purpose
trade_sltp	dict	Maps each trade's entry bar → detailed price levels and exit info.

7.2 Volatility-Based Stop-Loss Sizing

Important note: This is pure statistical mathematics — there is no machine learning or predictive model in this component. Stop-loss sizes are derived directly from the recent historical volatility of the asset using a rolling standard deviation.

Motivation

A fixed stop-loss (e.g. always 2% below entry) is naive because market volatility varies enormously. On a quiet day for a large-cap stock, a 2% move is a major event. For a cryptocurrency on a volatile day, a 2% move might happen in minutes as normal noise. A fixed SL that is too tight relative to normal volatility will be triggered constantly by noise, resulting in a long sequence of small losses even when the overall trade direction is correct. A volatility-adaptive SL places the stop just beyond the range of typical noise for that specific asset at that specific moment.

Calculation

At the moment a new trade is opened on bar i , the engine looks back at the most recent 20 closing prices (or fewer if less data is available) and computes their rolling standard deviation:

```
window = min(20, i)
pct_changes = Close[i-window : i].pct_change()
per_bar_vol = standard_deviation(pct_changes)
```

This gives a per-bar volatility expressed as a decimal fraction. For example, a `per_bar_vol` of 0.012 means the stock typically moves about 1.2% per bar.

Interval-specific bounds

A stop-loss that is too tight will be hit constantly by noise; one that is too wide means risking an impractical amount of capital. The `per_bar_vol` is clamped to sensible bounds that have been set specifically for each timeframe:

Interval	Minimum SL	Maximum SL	Rationale
1d	0.5%	10%	Daily moves rarely exceed 10% for major assets.
4h	0.3%	6%	4-hour candles cover half a trading session.
1h	0.2%	4%	Hourly moves are smaller than daily.
15m	0.1%	1%	Very short bars — extremely tight SL bounds.
5m	0.05%	0.5%	5-minute bars — minimal meaningful movement.

```
np.clip(per_bar_vol, min_sl, max_sl)
```

The final `sl_pct_val` is this clamped value. The SL price level is then:

```
sl_price = entry_price * (1 - sl_pct_val * direction)
```

For a Long (+1): $sl_price = entry \times (1 - sl_pct_val) \rightarrow$ below entry

```
For a Short (-1): sl_price = entry * (1 + sl_pct_val) → above entry
```


7.3 Position Sizing

Position sizing answers the question: *what fraction of total capital should be deployed in this trade?* The platform offers two modes.

Mode 1: Fixed Risk Per Trade (default)

The position size is calculated so that if the stop-loss is hit, the loss equals exactly `risk_per_trade` (default 2%) of total capital. This is sometimes called the '2% rule' and is one of the most widely used professional risk management frameworks.

```
pos_fraction = min( risk_per_trade / sl_pct_val, 1.0 )
```

Example: risk_per_trade=0.02 (2%), sl_pct_val=0.01 (1%).

pos_fraction = 0.02 / 0.01 = 2.0 → clamped to 1.0 (100% of capital, no leverage).

Example: risk_per_trade=0.02 (2%), sl_pct_val=0.04 (4%).

pos_fraction = 0.02 / 0.04 = 0.50 (50% of capital deployed).

When `sl_pct_val` is wide (high volatility), the position is sized down so the monetary risk remains constant. When `sl_pct_val` is tight (low volatility), the position is sized up, but never beyond 100%.

Mode 2: Volatility Targeting (optional)

When enabled, position sizes are calculated to target a specified level of annualised portfolio volatility (e.g. 15% per year) regardless of individual trade risk. This is a portfolio-level risk management approach used by many systematic hedge funds.

```
ann_vol = per_bar_vol × sqrt(periods_per_year)
```

```
pos_fraction = min( vol_target / ann_vol, 1.0 )
```

`per_bar_vol` is the same rolling 20-bar standard deviation used for SL sizing. `periods_per_year` converts it to an annualised figure (252 for daily, 1638 for hourly, etc). If the current annualised vol is 30% and the target is 15%, `pos_fraction = 15/30 = 0.5` — deploy 50% of capital.

During high-volatility periods, the strategy automatically reduces size; during low-volatility periods, it increases size.

This tends to produce more consistent risk-adjusted returns over time, but individual trade wins/losses can be larger than with fixed risk.

7.4 Take-Profit Calculation

The take-profit distance is always exactly twice the stop-loss distance. This means the reward-to-risk ratio (R:R) of every trade is 2:1 by design.

```
tp_pct_val = 2.0 × sl_pct_val
```

```
tp_price = entry_price × (1 + tp_pct_val × direction)
```

For a Long (+1): $tp_price = entry \times (1 + tp_pct_val) \rightarrow$ above entry

```
For a Short (-1): tp_price = entry × (1 - tp_pct_val) → below entry
```

Why 2:1?

A 2:1 reward-to-risk ratio means the strategy only needs to win 34% of trades to break even (before costs). For example: 10 trades, 3 winners (+2%) and 7 losers (-1%). $Net = 3 \times 2\% - 7 \times 1\% = 6\% - 7\% = -1\%$ (near breakeven at 30% win rate). Most strategies with genuine edge achieve win rates above 35%, making a 2:1 R:R profitable over time.

A fixed R:R also makes the strategy simple and transparent. More sophisticated systems use variable targets based on volatility projections or support/resistance levels, but these introduce additional complexity and more parameters to overfit.

7.5 Commission & Slippage

Both commission and slippage reduce the return of every trade by being subtracted at exit:

```
rt_cost = (2 × commission_pct + 2 × slippage_pct) × pos_fraction
```

```
strategy_return[exit bar] -= rt_cost
```

The factor of 2 accounts for both entry and exit legs of the trade. `commission_pct` covers broker fees (e.g. 0.05% per leg = 0.1% round trip). `slippage_pct` covers the cost of market impact — the fact that large orders move the price against the trader slightly at both entry and exit. Both are expressed as fractions of the position value, and both are scaled by `pos_fraction` so a smaller position incurs proportionally smaller costs.

7.6 Signal Re-Entry — The Pending Entry Mechanism

Because signals are event-based (fire on exactly one bar), a subtle re-entry problem must be handled carefully. The scenario is:

- Bar N: Long signal fires. Trade opens long.
- Bar M: Short signal fires. Long trade closes on bar M. At the same moment, a short trade should open.
- But: the short signal was on bar M. The bar M close is used as the exit price for the long trade. If we also enter short on bar M, we are entering and exiting on the same bar — possible in theory but requires careful logic.

The platform uses a **pending_entry** variable to queue the opposite direction for the next available bar:

- When a trade exits because the signal flipped, the new direction is saved as `pending_entry`.
- The `just_exited` flag is set to `True` for the current bar, preventing any new trade from opening immediately.
- On the very next bar (bar M+1), `pending_entry` is read and a new trade opens in the queued direction.
- Any new real signal arriving on a later bar overrides and clears `pending_entry`.

This correctly models the real-world constraint: you can't simultaneously exit one position and enter the reverse position at exactly the same price — there is at minimum a one-tick spread between the two fills.

8. Performance Metrics

The following metrics are computed after the backtest loop completes and displayed in the Backtest Results tab. Every metric is derived from the `strategy_returns` series (per-bar returns) and the `trade_log` DataFrame.

Total Return (%)

The cumulative compounded return of the strategy over the entire backtest period.

$$\text{Equity Curve}[-1] - 1.0$$

Where Equity Curve starts at 1.0 and compounds: $EC[t] = EC[t-1] \times (1 + r[t])$. A Total Return of 45.2% means £10,000 grew to £14,520 over the period.

Annualised Return (%)

The geometric annual growth rate — what average yearly return would compound to the total return over the period.

$$\text{Annualised Return} = (1 + \text{Total Return})^{(1 / \text{years})} - 1$$

`years` = number of bars / `periods_per_year`. This normalises returns across different-length backtests so they can be compared. A 3-year backtest with 50% total return annualises to approximately 14.5% per year.

Max Drawdown (%)

The largest peak-to-trough decline in the equity curve at any point during the backtest. It measures the worst loss experienced by an investor who bought at the worst possible moment.

$$\begin{aligned} \text{Rolling Peak}[t] &= \max(\text{Equity Curve}[0..t]) \\ \text{Drawdown}[t] &= (\text{Equity Curve}[t] - \text{Rolling Peak}[t]) / \text{Rolling Peak}[t] \\ \text{Max Drawdown} &= \min(\text{Drawdown}[t]) \text{ over all } t \end{aligned}$$

Max Drawdown is always negative (or zero). A Max Drawdown of -20% means the strategy at some point lost 20% of its peak value. This is the most important measure of downside risk — it represents the actual experience of an investor who held through the strategy's worst period.

Sharpe Ratio

The risk-adjusted return per unit of total volatility. It answers: how much return did the strategy earn per unit of risk taken? Higher is better; a ratio above 1.0 is generally considered good.

$$\text{Sharpe} = \text{mean}(\text{returns}) / \text{std}(\text{returns}) \times \text{sqrt}(\text{periods_per_year})$$

Returns here are per-bar strategy returns. The `sqrt(periods_per_year)` factor annualises the ratio so it is comparable across timeframes. The denominator is total volatility — both upside and downside moves. A Sharpe of 1.5 means the strategy earned 1.5 units of return for every unit of volatility. A Sharpe below 0 means the strategy lost money on average. Unusually high Sharpe ratios (above 3–4 on daily data) can indicate look-ahead bias or data errors.

Sortino Ratio

Like Sharpe, but penalises only downside volatility (negative returns) rather than total volatility. This is more appropriate for asymmetric return distributions — it doesn't penalise large upside moves the way Sharpe does.

$$\text{Downside Deviation} = \text{std}(\text{returns where returns} < 0) \quad \text{Sortino} = \frac{\text{mean}(\text{returns})}{\text{Downside Deviation} \times \sqrt{\text{periods_per_year}}}$$

A strategy with large winning trades and small losing trades will have a Sortino ratio significantly higher than its Sharpe ratio. The two together give a more complete picture of the risk/return profile.

Calmar Ratio

The annualised return divided by the maximum drawdown. It answers: how much annual return did the strategy earn per unit of maximum pain suffered?

$$\text{Calmar} = \frac{\text{Annualised Return}}{|\text{Max Drawdown}|}$$

A Calmar of 1.0 means the strategy earns one year of annual return for every point of max drawdown. A Calmar of 0.5 means it would take 2 years of gains to recover from the worst historical drawdown. Hedge funds typically target Calmar ratios above 0.5; ratios above 2.0 are exceptional.

Win Rate (%)

The percentage of completed trades that were profitable.

$$\text{Win Rate} = \frac{\text{Number of trades with Return} > 0}{\text{Total Trades}} \times 100$$

Note: this is trade-level win rate, not bar-level. A trade is profitable if its total compounded return across all bars it was open is greater than zero. With a 2:1 reward/risk ratio, a strategy needs a win rate above approximately 34% to be profitable before costs. Most systematic strategies win between 35% and 55% of trades.

Long Win Rate / Short Win Rate (%)

Win rates broken down separately for long and short trades. These reveal directional biases in the strategy's performance.

$$\text{Long Win Rate} = \frac{\text{Winning Long Trades}}{\text{Total Long Trades}} \times 100 \quad \text{Short Win Rate} = \frac{\text{Winning Short Trades}}{\text{Total Short Trades}} \times 100$$

For example, a strategy might have a 50% long win rate but only a 30% short win rate — suggesting the asset trends upward and short signals are less reliable. These metrics are only shown when at least some trades exist in that direction.

Exposure (%)

The percentage of total bars during which the strategy held an open position. A strategy with low exposure spends most of its time in cash.

$$\text{Exposure} = \frac{\text{bars where strategy_return} \neq 0}{\text{total bars}} \times 100$$

Low exposure (e.g. 20%) means the strategy is selective and only in the market 20% of the time. High exposure (e.g. 80%) means it is almost always in a position. Low exposure reduces risk but may also reduce

returns. Comparing the Sharpe ratio to exposure tells you whether the strategy earns its returns efficiently or just by being long a rising market most of the time.

Number of Trades

The total count of completed trades over the backtest period.

Count of rows in the trade log.

Statistical reliability requires a minimum number of trades to draw meaningful conclusions. With fewer than 20–30 trades, all metrics have very wide confidence intervals — the results could be due to luck rather than genuine strategy edge. The platform warns when fewer than 10 trades were generated.

9. Trade Log Columns

Every completed trade produces one row in the trade log. The columns are:

Column	Type	Description
Entry Date	Timestamp	The bar index (date/time) at which the trade was opened.
Exit Date	Timestamp	The bar at which the trade was closed.
Entry Price	float	The closing price of the bar just before entry — used as the fill price proxy.
SL Price	float	The stop-loss price level calculated at entry ($\text{entry} \times (1 \pm \text{sl_pct_val})$).
TP Price	float	The take-profit price level calculated at entry ($\text{entry} \times (1 \pm \text{tp_pct_val})$).
Exit Price	float	The actual exit fill price: <code>sl_price</code> if SL hit, <code>tp_price</code> if TP hit, or the close price if exited by signal or end of data.
Exit Reason	string	'SL' — stop-loss triggered. 'TP' — take-profit triggered. 'Signal' — a reversal signal caused the exit. 'End' — trade was open at the end of the data.
Direction	string	'Long' or 'Short'.
Return (%)	float	The compounded total return of the trade including all commission and slippage costs. Expressed as a percentage.
Bars Held	int	The number of bars the trade was open — from entry bar to exit bar.
MAE (%)	float	Maximum Adverse Excursion — the worst intra-trade drawdown from entry price, as a percentage. For a Long: $(\text{Entry} - \text{Trade Min Low}) / \text{Entry} \times 100$. Measures how far against you the trade went at its worst point.
MFE (%)	float	Maximum Favourable Excursion — the best intra-trade gain from entry price, as a percentage. For a Long: $(\text{Trade Max High} - \text{Entry}) / \text{Entry} \times 100$. Measures how far in your favour the trade went at its best point.

Using MAE and MFE together

- If most trades show large MFE but small final Return, the TP may be placed too close — the trade often went well past the TP before reversing.
- If MAE is frequently much larger than the final loss (SL exits), the SL may be placed too tight and triggering on normal noise.
- If $\text{MAE} \approx \text{final loss}$ for SL-exited trades, the SL is well-placed relative to the trade's actual adverse movement.
- A trade with MFE of 5% but Return of -1% suggests poor exit timing or an SL that was hit after a reversal from a profitable position.

10. Parameter Sweep

The Parameter Sweep tab runs a full backtest for every combination of two selected parameters across a configurable grid of values. The results are displayed as a table and as a Sharpe Ratio heatmap, making it easy to identify which parameter combinations perform best.

How it works

The sweep uses a Cartesian product of all parameter values. If you choose 4 steps for each of 2 parameters, the sweep runs $4 \times 4 = 16$ backtests. With 3 parameters at 4 steps each, it runs $4^3 = 64$ backtests.

For each combination, the strategy function is called with those parameter values to generate signals, `run_backtest` is called on those signals, and the resulting metrics are stored. All non-swept parameters use their default values from the strategy registry.

The linspace grid

Parameter values are generated using numpy's `linspace` — evenly spaced values between the parameter's minimum and maximum bounds. For integer parameters (like period lengths), the values are rounded to integers. For float parameters, they are rounded to 2 decimal places.

`n_steps=4` for integer param range `[2, 50]`: $\rightarrow [2, 18, 34, 50]$

`n_steps=4` for float param range `[0.5, 4.0]`: $\rightarrow [0.5, 1.5, 2.5, 4.0]$

Minimum trades filter

The `min_trades` slider (default 10) hides parameter combinations that generated fewer than the specified number of trades. This prevents the heatmap from highlighting parameter combinations that look excellent on paper but only executed 1 or 2 trades — which would be entirely due to luck.

The heatmap

After all backtests complete, the results are pivoted so that one parameter forms the X axis and the other forms the Y axis, with each cell coloured by the Sharpe Ratio. The colour scale runs from red (low/negative Sharpe) through yellow to green (high Sharpe).

Looking for smooth gradients in the heatmap rather than isolated peaks is important: a single bright green cell surrounded by red cells indicates overfitting — those exact parameter values happen to work on this historical data but are unlikely to generalise. A broad region of green cells indicates robust parameters that work well across a range of values.

Limitations and warnings

- **In-sample optimisation:** Any parameters selected using the sweep have been optimised on the same data used to evaluate them. This is called in-sample fitting. The metrics shown in the heatmap are optimistically biased — real out-of-sample performance will typically be worse.
- **Combinatorial explosion:** Adding a third parameter to the sweep makes it N^3 backtests. 8 steps $\times$ 8 steps $\times$ 8 steps = 512 backtests. The platform warns about this in the UI.

- **The sweep finds the past's best parameters:** Even a genuine robust region in the heatmap may not remain the best region in the future. Markets change regimes — a slow-period of 50 that worked for 3 years of trending market may fail in a subsequent ranging market.
- **Use as hypothesis generation, not final selection:** Use the sweep to understand how sensitive the strategy is to parameter choices, not to pick the single best combination and trust it blindly.

End of Documentation

Trading Research Platform — All strategies are for research and educational purposes only. Past performance does not guarantee future results.